Hardware Visualizer

Problem Context

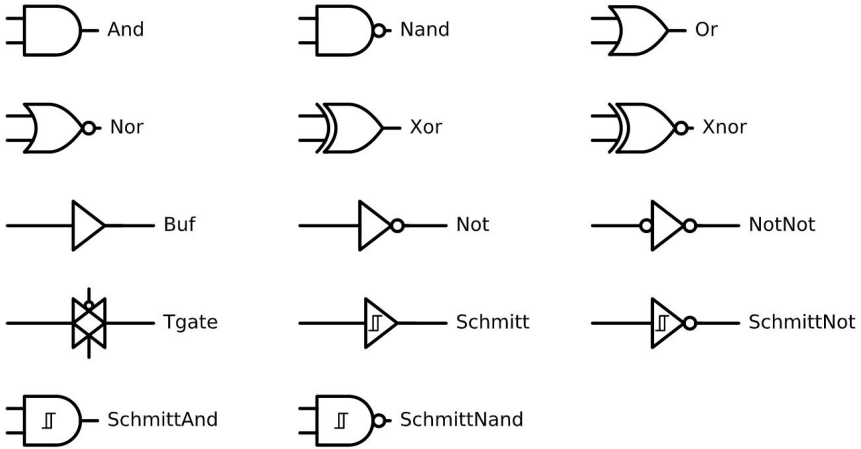
what the average Computer Science major is used to seeing:

```
out = not (a == b)
```

upon entering CSCE 312:

```
CHIP Xor {
    IN a, b;
    OUT out;

    PARTS:
    // Put your code here:
    Not(in=a, out=notA);
    Not(in=b, out=notB);
    And(a=notA, b=b, out=primis);
    And(a=a, b=notB, out=secundus);
    Or(a=primis, b=secundus, out=out);
}
```

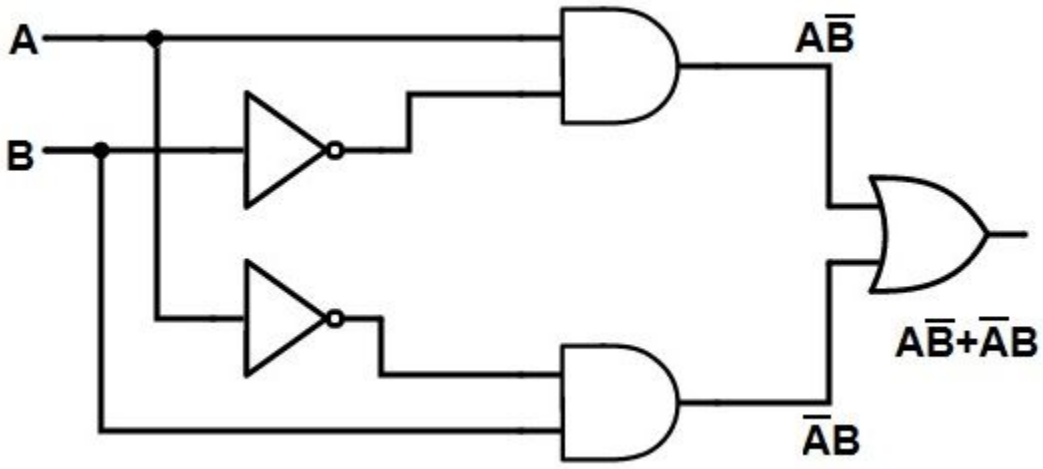
Using SchemDraw to visualize chips



→ **Part : Node**

→ **Signal Flow : Directed Edge**

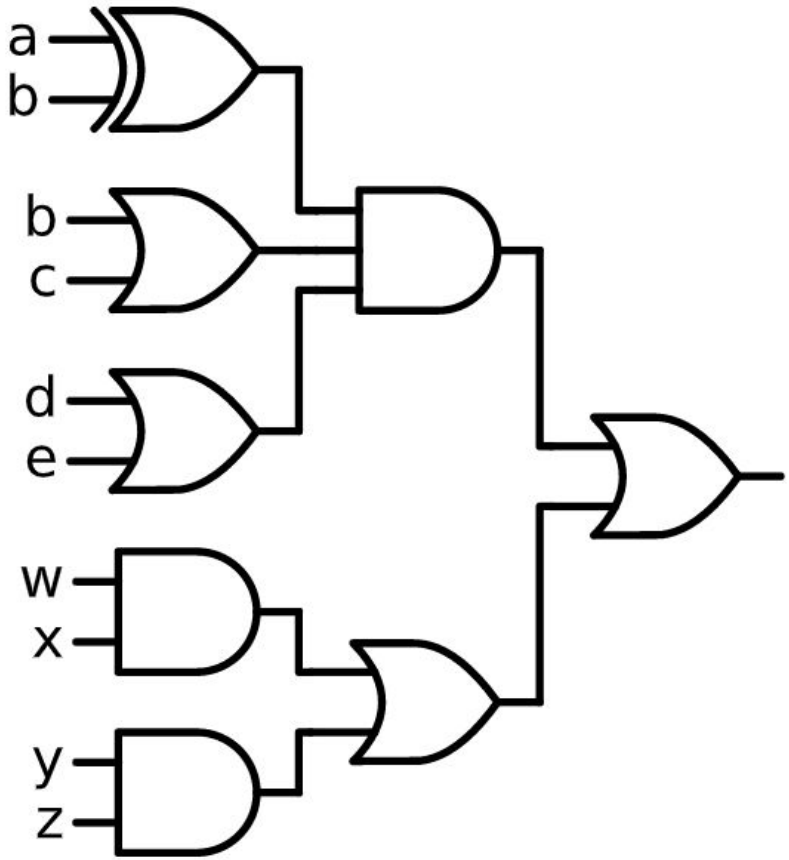
→ **Chip : Directed Graph**



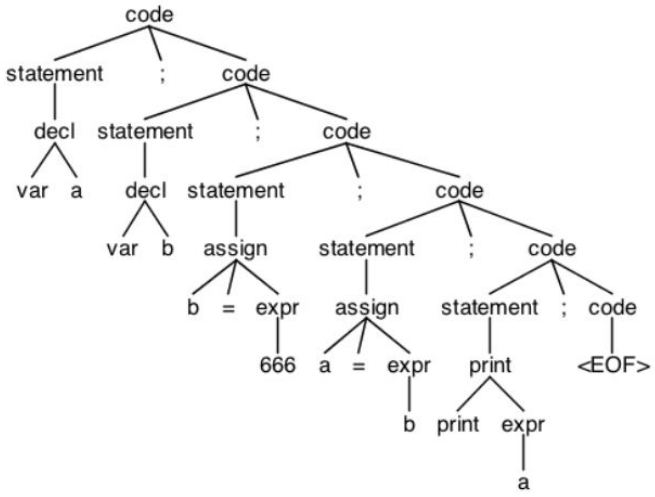
Initial Approach: build a digraph of chip parts and BFS Top.
Sort the parts so that they are drawn in a coherent order.

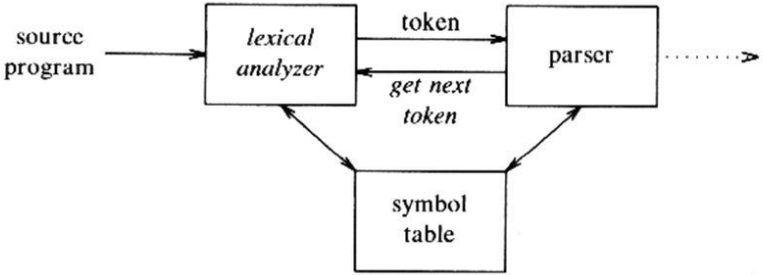
How do we decide *where* each part is drawn?

```
logicparse('((a xor b) and (b or c) and (d or e)) or ((w and x) or (y
```

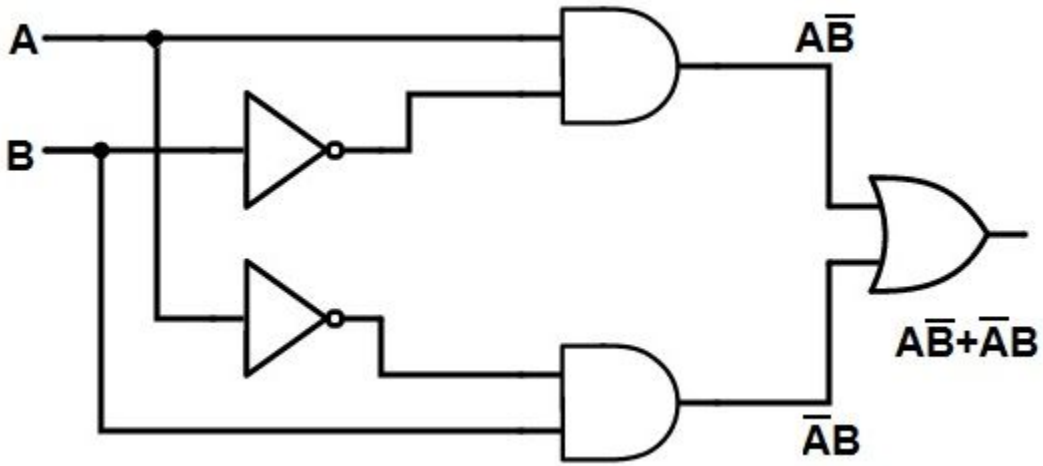


Revisiting Haskell: Syntax Trees (CSCE 314)






```
stringLiteral = char ('"') >>
                many stringChar >>= \s ->
                char ('"') >>
                whitespace >>
                return (Val (VString s))

stringChar = (char '\\' >> char 'n' >> return '\n')
             +++ sat (/= '"')
```

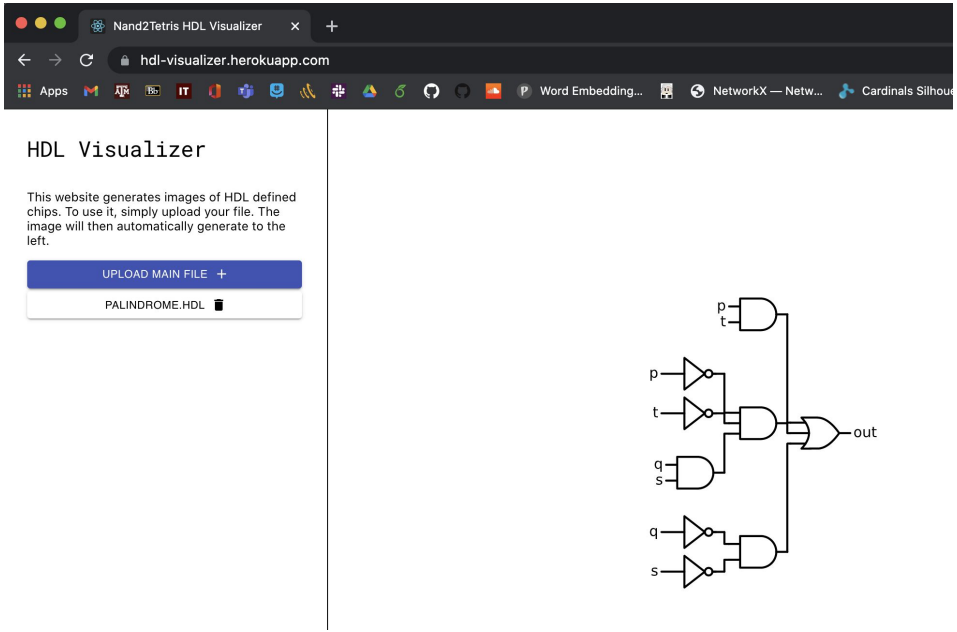


Implementation Approach: recursive evaluation of logic trees starting from output pins, then feed to SchemDraw's logic parser

```python
for op in outs:
    expr = ''

    terms = re.sub(r'[()]', '', op2expr[op]).split(' ')

    if len(terms) == 3:
        expr += str(reeval(terms[0]))
        expr += (str(' ') + str(terms[1]) + str(' '))
        expr += str(reeval(terms[2]))
    else:
        expr += str(terms[0])
        expr += str(reeval(terms[1]))

    feed.append(tuple([op, expr]))

return feed
```

```python
def reeval(pin):
    if pin not in op2expr.keys():
        return str(pin)

    subterms = re.sub(r'[()]', '', op2expr[pin]).split(' ')
    inputPinsInExpr = [n for n in subterms if n in ins]

    if not any(inputPinsInExpr):
        if len(subterms) == 3:
            subterms[0] = reeval(subterms[0])
            subterms[2] = reeval(subterms[2])
            return (str(subterms[0]) + str(' ') + str(subterms[1]) + str(' ') + str(subterms[2]))
        elif len(subterms) == 2:
            subterms[1] = reeval(subterms[1])
            return (str(subterms[0]) + str(' ') + str(subterms[1]))

    elif len(inputPinsInExpr) == 1:
        if len(subterms) == 3:
            if inputPinsInExpr[0] == subterms[0]:
                subterms[2] = reeval(subterms[2])
                return (str('(') + str(subterms[0]) + str(' ') + str(subterms[1]) + str(' ') + str(subterms[2]) + str(')'))
            else:
                subterms[0] = reeval(subterms[0])
                return (str('(') + str(subterms[0]) + str(' ') + str(subterms[1]) + str(' ') + str(subterms[2]) + str(')'))

        elif len(subterms) == 2:
            return (str('(') + str(subterms[0]) + str(' ') + str(subterms[1]) + str(')'))

    else: # 2 input pins, expression must have 3 terms
        return (str('(') + str(subterms[0]) + str(' ') + str(subterms[1]) + str(' ') + str(subterms[2]) + str(')'))
```

Final Product




```
CHIP Palindrome {

    IN p, q, r, s, t;
    OUT out;

    PARTS:
    // Put your code here:
    Not(in=p, out=notP);
    Not(in=q, out=notQ);
    Not(in=s, out=notS);
    Not(in=t, out=notT);

    And(a=p, b=t, out=PT);
    And(a=notP, b=notT, out=notPT);
    And(a=q, b=s, out=QS);
    And(a=notQ, b=notS, out=notQS);

    Or(a=PT, b=notPT, out=primis);
    Or(a=QS, b=notQS, out=secundus);

    And(a=primis, b=secundus, out=out);


}
```

How do we represent a Mux? DMux? Sequential Logic...